

# React Context API & useReducer

Advanced State Management in React

## 1 — Understanding Prop Drilling & Project Overview

When building React apps, you often need to pass data from a **parent component** down to a **deeply nested child**. If you pass data through many middle components that don't even use it, that is called **Prop Drilling**.

Example: App → Header → Navbar → UserAvatar. If only UserAvatar needs the user's name, you still have to pass it through Header and Navbar — that's prop drilling!

### → Component Composition as a Solution

**Component Composition** helps reduce prop drilling. Instead of passing props through every layer, you pass **components as children** (using `props.children`) or use **slot patterns** to place components directly where needed.

- You compose components together like puzzle pieces.
- The parent decides what to render inside the child component.
- This reduces unnecessary prop passing through middle components.

■ *Composition works well for UI structure, but for shared global state (like user login info), the Context API is a better choice.*

## 2 — Introducing the Context API

React's **Context API** is a built-in way to share data across the entire component tree **without prop drilling**. Think of it as a global storage that any component can access directly.

### → Creating & Providing The Context

To use Context, follow two steps:

- **Step 1 – Create:** Use `React.createContext()` to create a context object.
- **Step 2 – Provide:** Wrap your components with the `<Context.Provider>` tag and pass a **value** prop.

```
const ThemeContext = React.createContext(); <ThemeContext.Provider value={{ theme: 'light' }}> <App /> </ThemeContext.Provider>
```

### → Consuming the Context

Any child component inside the Provider can **consume (read)** the context value using the `useContext` hook.

```
const { theme } = useContext(ThemeContext);
```

- No need to pass props — the component gets the value directly from context.

### → Linking the Context to State

Context becomes powerful when you **link it to state**. Store your state in the parent/provider component and pass both the **state value** and **state updater** function through context.

```
const [user, setUser] = useState(null); <UserContext.Provider value={{ user, setUser }}> <App /> </UserContext.Provider>
```

### → A Different Way Of Consuming Context

Besides `useContext`, you can also consume context using the **Consumer component** pattern (older approach):

```
<ThemeContext.Consumer> {(value) => <div>Theme is: {value.theme}</div>}
</ThemeContext.Consumer>
```

■ *The `useContext` hook is cleaner and more popular in modern React code.*

### → What Happens When Context Values Change?

Whenever the **value** passed to a Provider changes, **all components** that consume that context will **automatically re-render** with the new value.

- This makes it easy to keep UI in sync with shared state.
- Be careful — too many re-renders can slow down your app if context is overused.
- Split contexts by concern (e.g. `AuthContext`, `ThemeContext`) to limit re-renders.

### → Outsourcing Context & State Into a Separate Provider Component

It is a **best practice** to move your context creation and state logic into a **separate file**. This keeps your code clean and easy to maintain.

```
// useContext.js export const useContext = createContext(); export function
UserProvider({ children }) { const [user, setUser] = useState(null); return (
<UserContext.Provider value={{ user, setUser }}> {children} </UserContext.Provider> );
}
```

Then in your main file, just wrap the app with `<UserProvider>`:

```
<UserProvider> <App /> </UserProvider>
```

## 3 — Introducing the `useReducer` Hook

The `useReducer` hook is an alternative to `useState`. It is best when your state logic is **complex** — for example, when the next state depends on the previous one, or when you have many related state variables.

It works like this: instead of directly setting state, you **dispatch an action**. A **reducer function** receives the current state and the action, then returns the **new state**.

| Term     | What it means                                                   |
|----------|-----------------------------------------------------------------|
| state    | The current data (e.g. a counter value)                         |
| action   | An object describing what happened (e.g. { type: 'increment' }) |
| reducer  | A function that takes (state, action) and returns new state     |
| dispatch | A function you call to send an action to the reducer            |

### → Dispatching Actions & Editing State with `useReducer`

Here is a simple counter example using `useReducer`:

```
const initialState = { count: 0 }; function reducer(state, action) { if (action.type
=== 'increment') { return { count: state.count + 1 }; } if (action.type ===
'decrement') { return { count: state.count - 1 }; } return state; } function Counter()
{ const [state, dispatch] = useReducer(reducer, initialState); return ( <div> <p>Count:
{state.count}</p> <button onClick={() => dispatch({ type: 'increment' })}>+</button>
<button onClick={() => dispatch({ type: 'decrement' })}>-</button> </div> ); }
```

Key steps:

- **Define reducer:** A function that handles each action type and returns updated state.
- **Call useReducer:** Pass the reducer and initial state. Get back [state, dispatch].
- **Dispatch an action:** Call dispatch({ type: 'actionName' }) to trigger state changes.

■ Combine useReducer with Context API for powerful global state management — similar to how Redux works, but with zero extra libraries!

### Quick Summary

| Concept       | Purpose                           | Key Benefit                      |
|---------------|-----------------------------------|----------------------------------|
| Prop Drilling | Passing props through many layers | Understand why Context is needed |
| Composition   | Pass components as children       | Reduces unnecessary props        |
| createContext | Create a shared data store        | One-time setup per feature       |
| Provider      | Wrap components to share data     | All children can access value    |
| useContext    | Read context in any component     | Clean, no prop chains needed     |
| useReducer    | Manage complex state logic        | Predictable state updates        |